

Integrantes do Grupo:

João Gabriel Ribeiro Holanda

João Victor Fernandes Lima

Luan Tadeu Lima Rezende Dias

Compilador BRL — Documentação Formal

Trabalho Prático — Compiladores

Ciência da Computação — Dom Helder / Escola Superior

Prof. Dr. Marcos W. Rodrigues

I — Autômato Finito Determinístico (AFD)

1.1 — AFD do Analisador Léxico

Definição formal: $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}, q_{12}, q_{13}, q_{14}\}$
- $\Sigma = \{\text{letra, dígito, sublinhado, espaço, +, -, *, /, <, >, =, \&, :, ;, ,, (,), ", .}\} \cup \{\text{CR, LF}\}$
- q_0 = estado inicial
- $F = \{q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}, q_{12}, q_{13}, q_{14}\}$ (estados finais de aceitação)

1.2 — Tabela de Transições

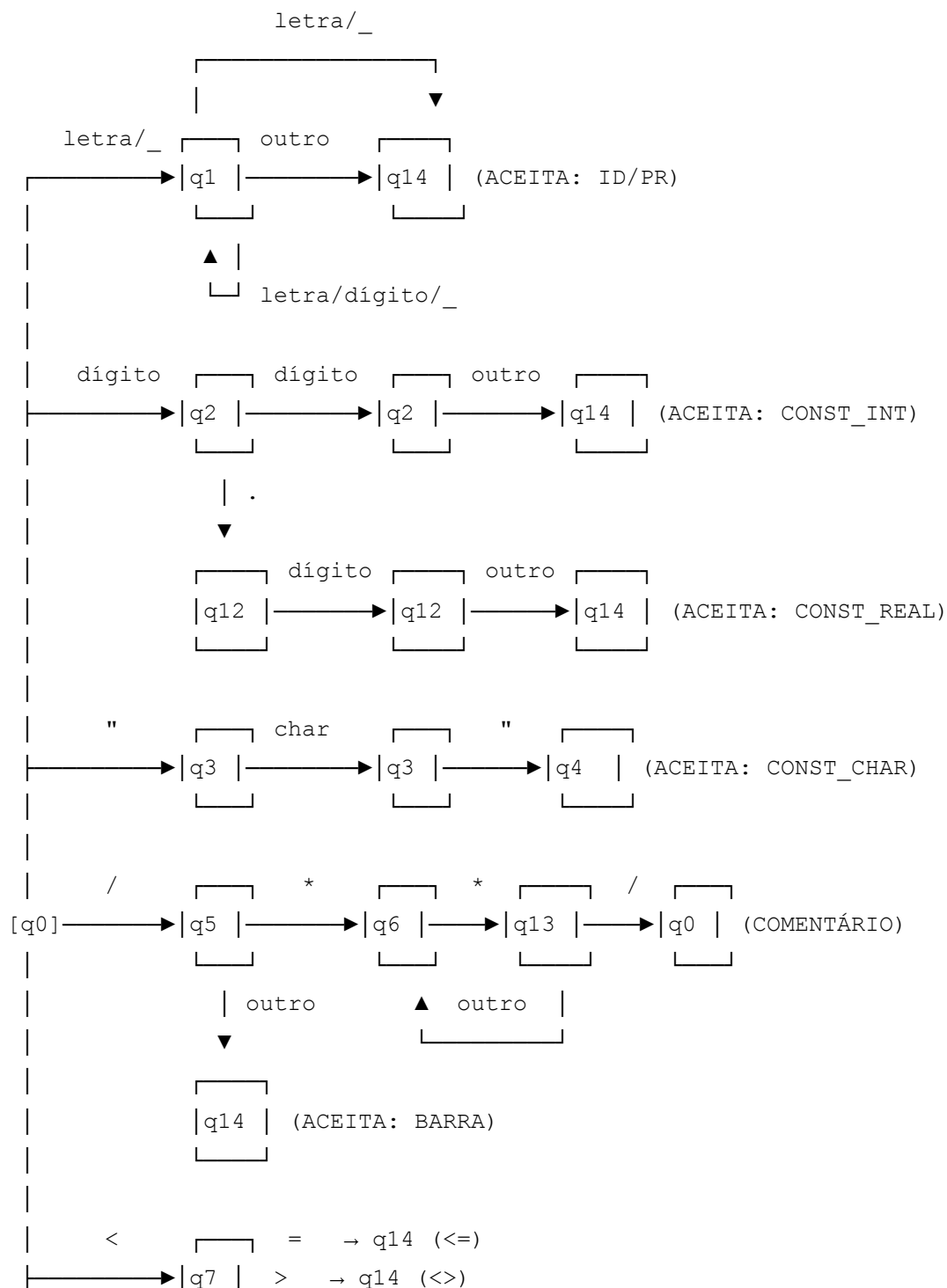
Estado	Entrada	Próximo Estado / Ação
q0	espaço, tab, CR, LF	q0 (ignorar)
q0	letra, sublinhado	q1 (início identificador/palavra reservada)
q0	dígito	q2 (início número inteiro)
q0	"	q3 (início string/caractere)

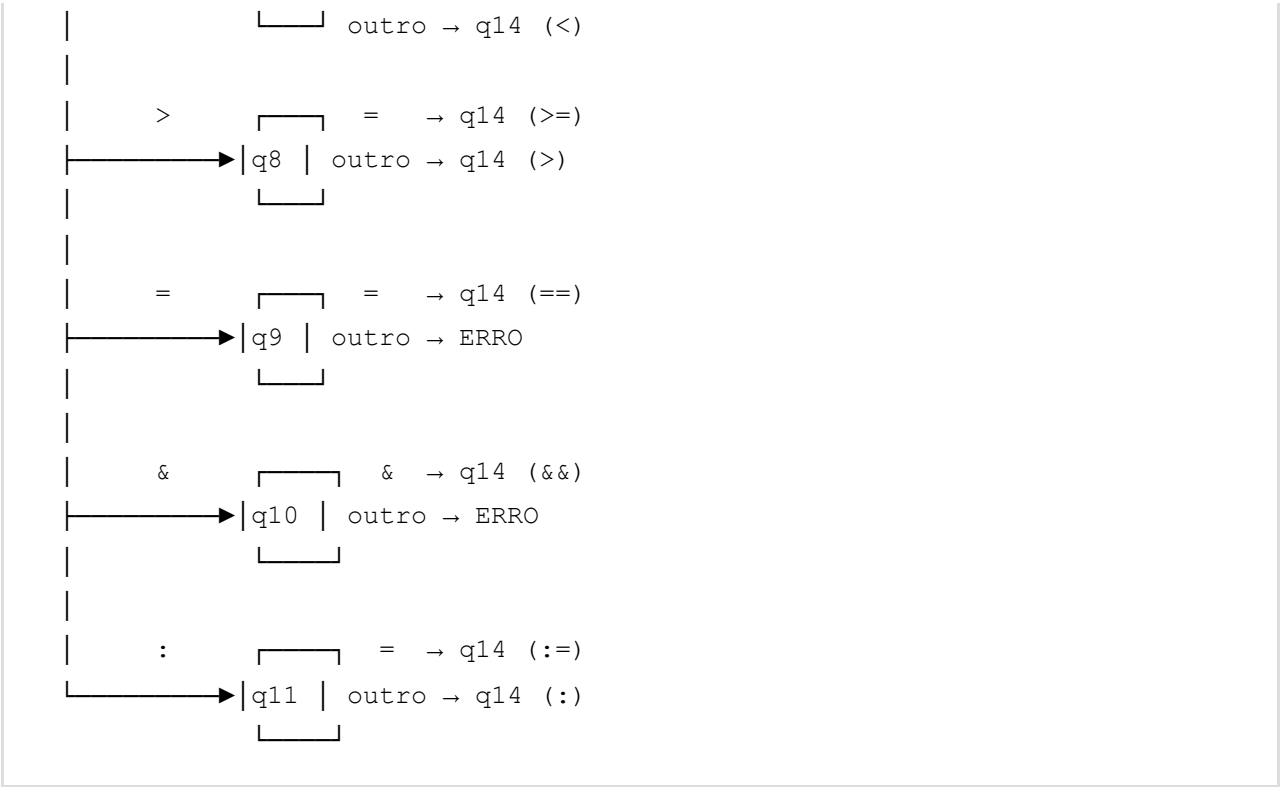
q0	/	q5 (pode ser divisão ou comentário)
q0	+	q14 → token MAIS
q0	-	q14 → token MENOS
q0	*	q14 → token VEZES
q0	<	q7 (pode ser <, <=, <>)
q0	>	q8 (pode ser >, >=)
q0	=	q9 (deve vir outro =)
q0	&	q10 (deve vir outro &)
q0	:	q11 (pode ser : ou :=)
q0	;	q14 → token PONTO_VIRGULA
q0	,	q14 → token VIRGULA
q0	(	q14 → token ABRE_PAREN
q0	)	q14 → token FECHA_PAREN
q0	outro	ERRO LÉXICO
q1	letra, dígito, _	q1 (continua identificador)
q1	outro	q14 → token ID ou PALAVRA_RESERVADA
q2	dígito	q2 (continua inteiro)
q2	.	q12 (início parte decimal → real)
q2	outro	q14 → token CONST_INTEIRO
q12	dígito	q12 (continua decimal)
q12	outro	q14 → token CONST_REAL
q3	qualquer exceto "	q3 (continua string)

		e exceto CR/LF	
q3	"		q4 → token CONST_CARACTERE
q3	CR ou LF		ERRO LÉXICO (quebra de linha em string)
q3	EOF		ERRO LÉXICO (string não fechada)
q5	*		q6 (início comentário /*)
q5	outro		q14 → token BARRA (divisão)
q6	*		q13 (possível fim de comentário)
q6	outro		q6 (dentro do comentário)
q6	EOF		ERRO LÉXICO (comentário não fechado)
q13	/		q0 (fim do comentário, volta ao início)
q13	*		q13 (continua tentando fechar)
q13	outro		q6 (volta para dentro do comentário)
q7	=		q14 → token MENOR_IGUAL (<=)
q7	>		q14 → token DIFERENTE (<>)
q7	outro		q14 → token MENOR (<)
q8	=		q14 → token MAIOR_IGUAL (>=)
q8	outro		q14 → token MAIOR (>)
q9	=		q14 → token IGUAL (==)
q9	outro		ERRO LÉXICO (= isolado)
q10	&		q14 → token E_LOGICO (&&)

q10	outro	ERRO LÉXICO (& isolado)
q11	=	q14 → token ATRIBUICAO (:=)
q11	outro	q14 → token DOIS_PONTOS (:)

1.3 — Diagrama do AFD (Representação Textual)





1.4 — Palavras Reservadas

As seguintes palavras são reservadas e reconhecidas pelo AFD no estado q1 (após identificar que o lexema é uma palavra reservada via tabela de lookup):

Palavra	Token
inteiro	INTEIRO
caractere	CARACTERE
logico	LOGICO
real	REAL
se	SE
entao	ENTAO
senao	SENAO
enquanto	ENQUANTO
faca	FACA
leitura	LEITURA
escrita	ESCRITA
ou	OU
e	E

Palavra	Token
nao	NAO
inicio	INICIO
fim	FIM
div	DIV
mod	MOD
verdadeiro	VERDADEIRO
falso	FALSO

II — Gramática Formal do Compilador BRL

2.1 — Gramática Livre de Contexto (BNF)

<programa>	→ 'inicio' <identificador> ';' <declaracoes> <instrucoes> 'fim'
<declaracoes>	→ <declaracao> ';' <declaracoes> ε
<declaracao>	→ <id_list> ':' <tipo>
<id_list>	→ <identificador> ',' <id_list> <identificador>
<tipo>	→ 'inteiro' 'caractere' 'logico' 'real'
<instrucoes>	→ <instrucao> ';' <instrucoes> <instrucao> ε
<instrucao>	→ <atribuicao> <se> <enquanto> <leitura> <escrita>
<atribuicao>	→ <identificador> ':=' <expressao>

```

<se>                → 'se' <expressao> 'entao' 'inicio' <instrucoes> 'fim'
                    | 'se' <expressao> 'entao' 'inicio' <instrucoes> 'fim'
                      'senao' 'inicio' <instrucoes> 'fim'

<enquanto>          → 'enquanto' <expressao> 'faca' 'inicio' <instrucoes> 'fim'

<leitura>            → 'leitura' '(' <id_list> ')'

<escrita>            → 'escrita' '(' <expressao> ')'

<expressao>          → <exp_simples> <relop> <exp_simples>
                    | <exp_simples>

<exp_simples>        → <termo> <addop> <exp_simples>
                    | <termo>

<termo>              → <fator> <mulop> <termo>
                    | <fator>

<fator>              → <identificador>
                    | <constante>
                    | '(' <expressao> ')'
                    | 'nao' <fator>
                    | '+' <fator>
                    | '-' <fator>

<constante>          → <const_inteiro>
                    | <const_real>
                    | <const_caractere>
                    | 'verdadeiro'
                    | 'falso'

<relop>              → '==' | '<' | '<' | '>' | '<=' | '>='

<addop>              → '+' | '-' | 'ou'

<mulop>              → '*' | '/' | 'div' | 'mod' | '&&'

```

2.2 — Definições Léxicas (Expressões Regulares)

```

<identificador>      → (letra | '_') (letra | dígito | '_')*

<const_inteiro>      → dígito+

```

```

<const_real>      → dígito+ '.' dígito+

<const_caractere> → ''' (caractere_válido)* '''

<letra>           → [a-zA-Z]

<dígito>          → [0-9]

<caractere_válido> → qualquer caractere exceto ''' , CR e LF

```

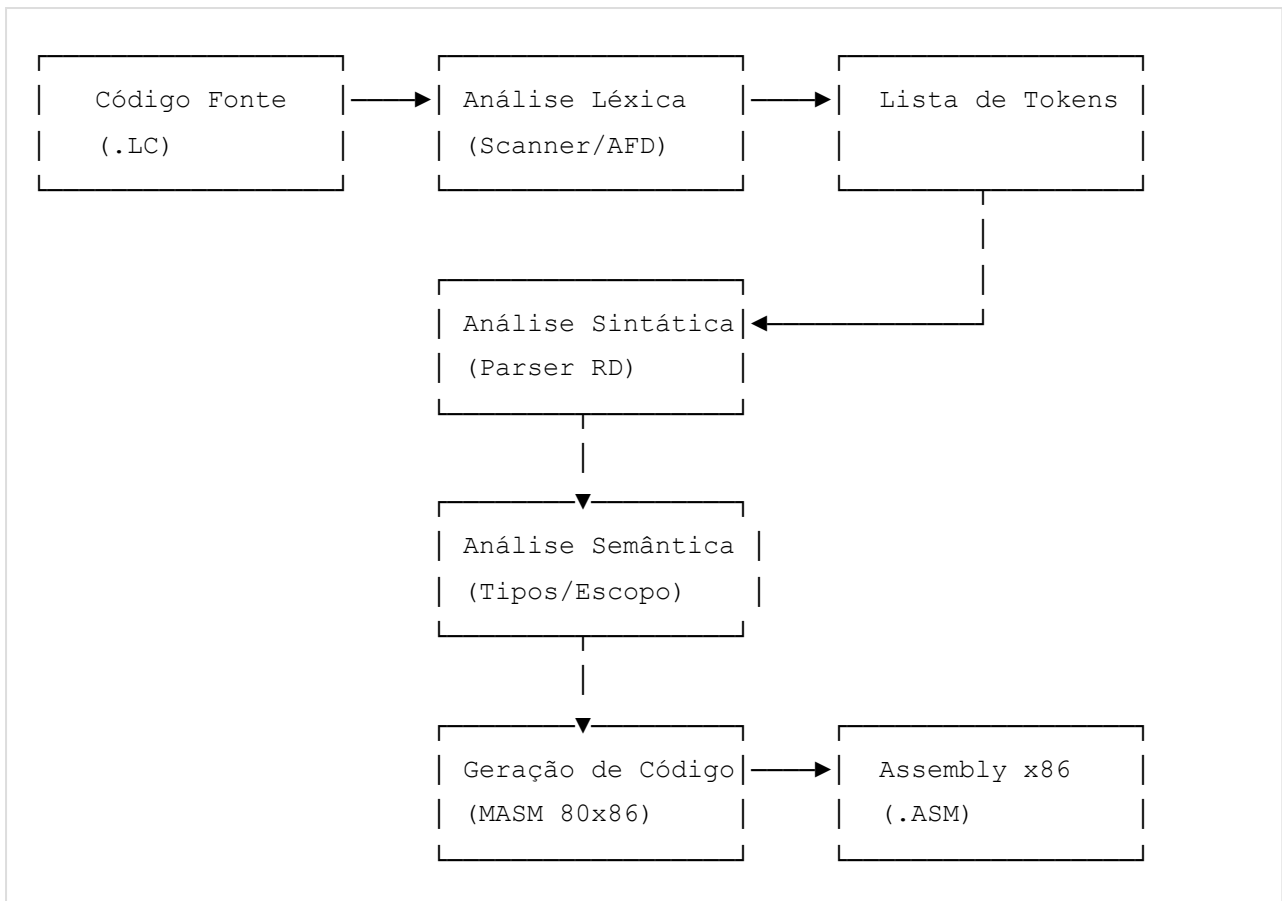
2.3 — Precedência de Operadores (da maior para a menor)

Prioridade	Operadores	Associatividade
1	()	—
2	nao, + (unário), - (unário)	Direita
3	*, /, div, mod, &&	Esquerda
4	+, -, ou	Esquerda
5	==, <, <=, >, >=	Esquerda

2.4 — Regras Semânticas

Regra	Descrição
S1	Todo identificador deve ser declarado antes do uso
S2	Não pode haver declaração duplicada do mesmo identificador
S3	Atribuição: tipo da expressão deve ser compatível com tipo da variável
S4	Promoção de tipo: inteiro pode ser atribuído a real
S5	Operadores aritméticos (+, -, *, /) exigem operandos numéricos (inteiro/real)
S6	Operadores div e mod exigem operandos inteiros
S7	Operadores lógicos (&&, ou, nao) exigem operandos lógicos
S8	Operadores relacionais exigem operandos numéricos (exceto == entre caracteres)
S9	Concatenação (+) entre caracteres resulta em caractere
S10	Condição de 'se' e 'enquanto' deve ser do tipo lógico

III — Arquitetura do Compilador



Front-end:

- `AnalizadorLexico.java` — Scanner baseado no AFD
- `AnalizadorSintatico.java` — Parser descendente recursivo
- `AnalizadorSemantico.java` — Verificação de tipos e declarações

Back-end:

- `GeradorCodigo.java` — Geração de Assembly MASM x86

Estruturas auxiliares:

- `Token.java` / `TokenType.java` — Representação de tokens
- `TabelaSimbolos.java` — Tabela de símbolos com endereços de memória